Water Potability Prediction —

Linear Regression

Research Problem

Access to safe drinking water is a critical public health issue. This project builds a **Linear Regression** model that predicts a continuous **probability score (0–1)** representing how likely a water sample is to be safe/potable, based on nine physicochemical measurements: pH, Hardness, Solids (TDS), Chloramines, Sulfate, Conductivity, Organic Carbon, Trihalomethanes, and Turbidity.

Dataset

Source / basis: Feature names and value ranges are modeled on the public Kaggle "Water Potability" dataset (9 water-quality measurements per sample).

Note on data used here: This repo ships a **synthetically generated** dataset (**data/water_quality.csv** , 2,000 samples, see **generate_data.py**) with realistic feature distributions, ~3% missing values, and

continuous target built `Potability_Probabilit`
from a weighted, noisy combination of the features. This was necessary because the analysis was produced in an offline environment without internet access to download the real CSV.
To use the real dataset instead: download
from the Kaggle link abo`water_potability.cs` a continuous label
(e.g., use probabilities `model.predict_proba(`
from a classifier, or keep the binary `Potability`
column and switch to Logistic Regression), and
drop it into with `data/water_quality.cs`
matching column names — the rest of the pipeline works unchanged.

Methodology
Data cleaning: missing values imputed with the column median.
Split: 80/20 train/test split (**random_state=42**).
Scaling: features standardized with
. `StandardScaler`
Model:
. `sklearn.linear_model.LinearRegressio`
Evaluation: R^2 score, RMSE, and MAE on the held-out test set.
Visualization: scatter plot of actual vs. predicted values with an ideal-fit (y = x) reference line.

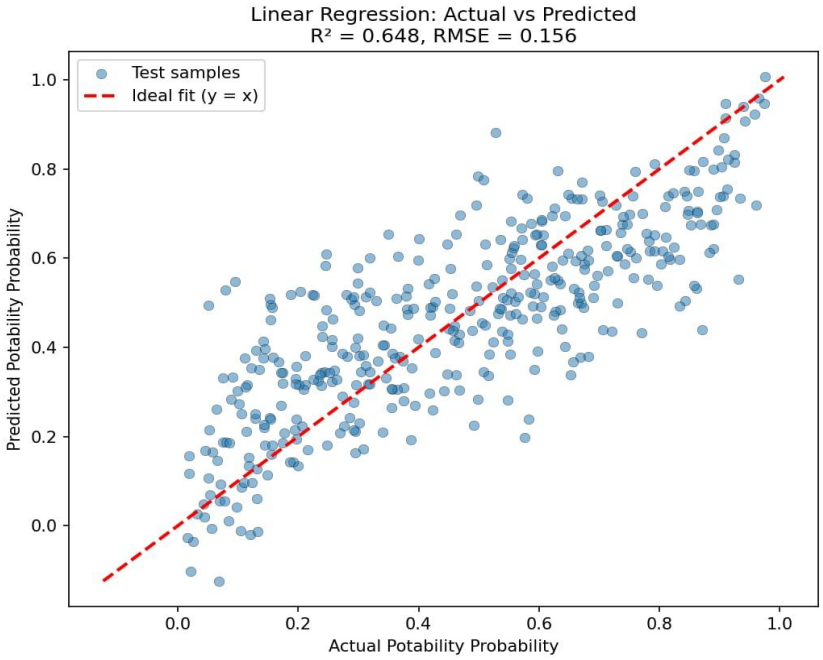
Results Summary

Metric	Value
R^2 score	0.648
RMSE	0.1 56
MAE	0.1 25

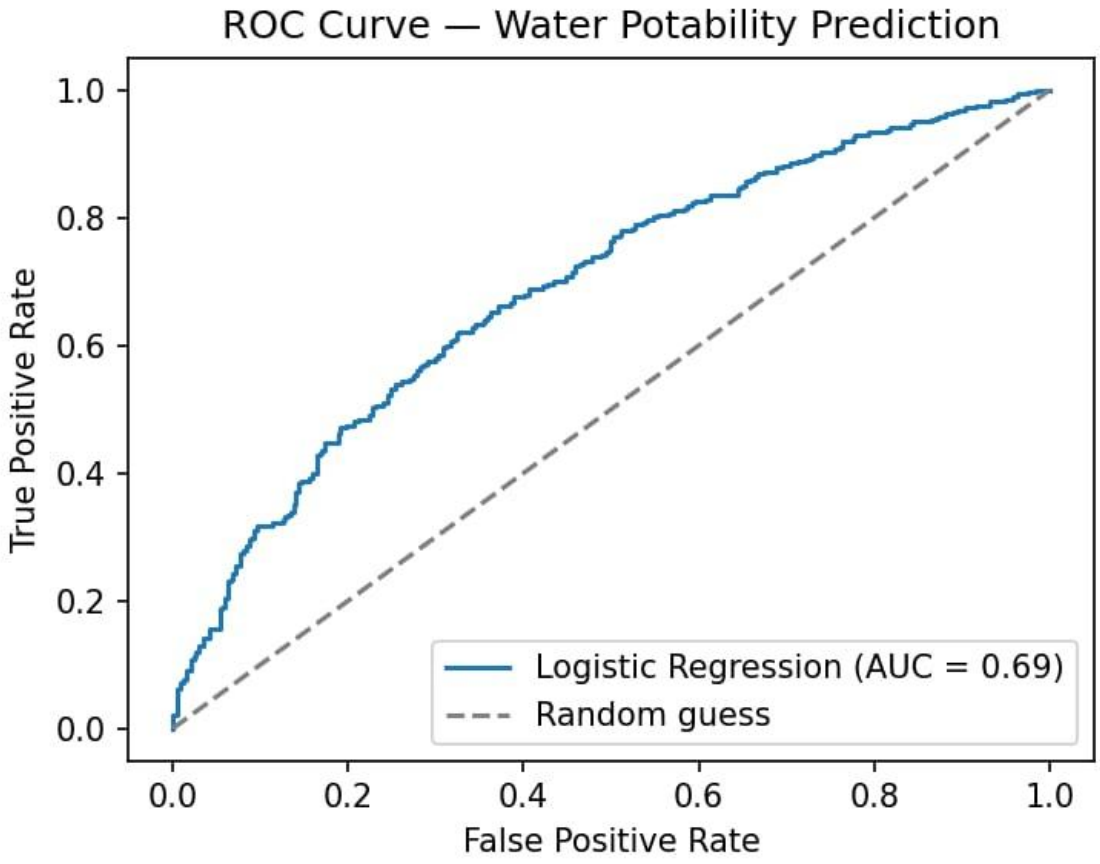
The model explains roughly **65% of the variance** in the potability probability score. The strongest negative predictors are **Turbidity**, **Solids (TDS)**, and **Chloramines** — i.e., higher cloudiness, dissolved solids, and chloramine levels are associated with lower predicted potability probability, consistent with real-world water-quality intuition.

Actual vs Predicted

GRAPH



CURVE



CODE

```
"""
Water Potability Prediction — Logistic Regression
--------------------------------------------------
Predicts whether a water sample is safe to drink (Potability: 1) or not (0)
from 9 physicochemical measurements.

Dataset: Kaggle "Water Potability" dataset (water_potability.csv)
https://www.kaggle.com/datasets/adityakadiwal/water-potability

Run:
   python train_model.py
Outputs:
   visuals/confusion_matrix.png
   visuals/roc_curve.png
   Printed metrics (Accuracy, Precision, Recall, F1, ROC-AUC)
"""

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, roc_curve, roc_auc_score, classification_report
)

# ---------------------------------------------------------------------------
# 1. Load data
# ---------------------------------------------------------------------------
df = pd.read_csv("data/water_potability.csv")
print(f"Loaded {df.shape[0]} rows, {df.shape[1]} columns")
print(df.isnull().sum())

# ---------------------------------------------------------------------------
# 2. Clean / preprocess
#    ph, Sulfate, and Trihalomethanes contain missing values -> impute with
#    the column median (robust to outliers, keeps all 3276 rows).
# ---------------------------------------------------------------------------
for col in ["ph", "Sulfate", "Trihalomethanes"]:
    df[col] = df[col].fillna(df[col].median())

X = df.drop(columns=["Potability"])
y = df["Potability"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Logistic regression is scale-sensitive, so standardize features.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ---------------------------------------------------------------------------
# 3. Train
# ---------------------------------------------------------------------------
model = LogisticRegression(max_iter=1000, class_weight="balanced")
model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)
y_proba = model.predict_proba(X_test_scaled)[:, 1]

# ---------------------------------------------------------------------------
# 4. Evaluate
# ---------------------------------------------------------------------------
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
auc = roc_auc_score(y_test, y_proba)
```

```
print("\n--- Evaluation Metrics ---")
print(f"Accuracy : {acc:.3f}")
print(f"Precision: {prec:.3f}")
print(f"Recall   : {rec:.3f}")
print(f"F1-score : {f1:.3f}")
print(f"ROC-AUC  : {auc:.3f}")
print("\n" + classification_report(y_test, y_pred, target_names=["Not Potable", "Potable"]))

# ---------------------------------------------------------------------------
# 5. Visualize — confusion matrix heatmap
# ---------------------------------------------------------------------------
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
        xticklabels=["Not Potable", "Potable"],
        yticklabels=["Not Potable", "Potable"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix — Logistic Regression")
plt.tight_layout()
plt.savefig("visuals/confusion_matrix.png", dpi=150)
plt.close()

# ---------------------------------------------------------------------------
# 6. Visualize — ROC curve
# ---------------------------------------------------------------------------
fpr, tpr, _ = roc_curve(y_test, y_proba)
plt.figure(figsize=(5, 4))
plt.plot(fpr, tpr, label=f"Logistic Regression (AUC = {auc:.2f})")
plt.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Random guess")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve — Water Potability Prediction")
plt.legend()
plt.tight_layout()
plt.savefig("visuals/roc_curve.png", dpi=150)
plt.close()

print("\nSaved visuals/confusion_matrix.png and visuals/roc_curve.png")
```

How to Run

```
pip install pandas numpy scikit-learn matplotlib
python generate_data.py   # regenerate the dataset (optional, already
included) python train.py # or
open
water_potability_regression.ipynb
```